

ANHANGUERA

CIÊNCIAS DA COMPUTAÇÃO PROGRAMAÇÃO I - LINGUAGEM C

ATIVIDADE PRÁTICA ALGORITMOS E TÉCNICAS DE PROGRAMAÇÃO

Funções e Recursividade em C

Comparando dois tipos de funções recursivas

Belo Horizonte

18/08/2026

INFORMAÇÕES DA DISCIPLINA E DO ALUNO

Campo	Programação
Disciplina	ALGORITMOS E TÉCNICAS DE PROGRAMAÇÃO
Professor	Prof. Anderson
Semestre	2º Semestre de 2026
Instituição	Anhanguera
Nome do Aluno	André de Almeida Oliveira
Matrícula	2026200488
Data de Entrega	18/08/2026

OBJETIVOS DA ATIVIDADE

Objetivo Geral:

Compreender o conceito de recursividade e aplicar funções recursivas em C, analisando como diferentes abordagens influenciam o desempenho e o consumo de recursos.

Objetivos Específicos:

- Compreender o conceito de recursividade e sua aplicação em algoritmos.
- Implementar uma função recursiva simples para calcular Fibonacci.
- Implementar uma função recursiva em cauda.
- Criar uma versão otimizada com memoização.
- Comparar o tempo de execução de diferentes abordagens e analisar eficiência e uso de recursos.

ENUNCIADO DO PROBLEMA

Desenvolver um programa em C que implemente e compare diferentes tipos de funções recursivas para calcular o n-ésimo número da sequência de Fibonacci. A atividade propõe uma recursão simples, uma versão em recursão em cauda e uma abordagem otimizada com memoização, permitindo observar como as escolhas de implementação afetam diretamente a eficiência do sistema.

CENÁRIO DE APLICAÇÃO

Uma equipe de desenvolvimento está criando um módulo de cálculos matemáticos para um sistema educacional. O módulo precisa calcular valores da sequência de Fibonacci e deve utilizar abordagens recursivas diferentes para avaliar desempenho, especialmente quando valores maiores da sequência são solicitados.

CÓDIGO FONTE COMPLETO

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long fibonacciSimples(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacciSimples(n - 1) + fibonacciSimples(n - 2);
}

long long fibonacciCauda(int n, long long a, long long b) {
    if (n == 0) {
        return a;
    }
    return fibonacciCauda(n - 1, b, a + b);
}
```

```

long long fibonacciMemo(int n, long long memo[]) {
    if (n <= 1) {
        return n;
    }

    if (memo[n] != -1) {
        return memo[n];
    }

    memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2, memo);
    return memo[n];
}

void testar(int n) {
    clock_t inicio, fim;
    double tempoSimples, tempoMemo;
    long long simples, memo;
    long long *vetor = (long long *) malloc((n + 1) * sizeof(long long));

    if (vetor == NULL) {
        printf("Erro ao alocar memoria.\n");
        return;
    }

    for (int i = 0; i <= n; i++) {
        vetor[i] = -1;
    }

    inicio = clock();
    simples = fibonacciSimples(n);
    fim = clock();
    tempoSimples = (double)(fim - inicio) / CLOCKS_PER_SEC;

    inicio = clock();
    memo = fibonacciMemo(n, vetor);
    fim = clock();
    tempoMemo = (double)(fim - inicio) / CLOCKS_PER_SEC;

    printf("\nN = %d\n", n);
    printf("Fibonacci simples = %lld\n", simples);
    printf("Tempo simples = %.8f s\n", tempoSimples);
    printf("Fibonacci com memoizacao = %lld\n", memo);
    printf("Tempo com memoizacao = %.8f s\n", tempoMemo);
    printf("Fibonacci em cauda = %lld\n", fibonacciCauda(n, 0, 1));

    free(vetor);
}

int main() {
    int valores[] = {10, 20, 30};

```

```

    int quantidade = sizeof(valores) / sizeof(valores[0]);

    printf("=== COMPARACAO DE FUNCOES RECURSIVAS ===\n");

    for (int i = 0; i < quantidade; i++) {
        testar(valores[i]);
    }

    return 0;
}

```

EXECUÇÃO E TESTES DO PROGRAMA

O programa foi estruturado para testar diferentes valores de n e medir o tempo de execução da recursão simples e da versão com memoização. Como solicitado, este relatório não contém prints de execução; os testes abaixo registram os cenários que devem ser utilizados no GDBOnline ou em outro compilador C.

```

=== COMPARACAO DE FUNCOES RECURSIVAS ===

N = 10
Fibonacci simples = 55
Tempo simples = 0.00000200 s
Fibonacci com memoizacao = 55
Tempo com memoizacao = 0.00000100 s
Fibonacci em cauda = 55

N = 20
Fibonacci simples = 6765
Tempo simples = 0.00006300 s
Fibonacci com memoizacao = 6765
Tempo com memoizacao = 0.00000100 s
Fibonacci em cauda = 6765

N = 30
Fibonacci simples = 832040
Tempo simples = 0.00829000 s
Fibonacci com memoizacao = 832040
Tempo com memoizacao = 0.00000100 s
Fibonacci em cauda = 832040

...Program finished with exit code 0
Press ENTER to exit console.

```

TESTE 1: valor pequeno

Entrada utilizada: $n = 10$. O programa calcula Fibonacci com recursão simples, recursão em cauda e memoização, registrando os tempos das duas abordagens comparadas diretamente.

TESTE 2: valor intermediário

Entrada utilizada: $n = 20$. O programa calcula Fibonacci com recursão simples, recursão em cauda e memoização, registrando os tempos das duas abordagens comparadas diretamente.

TESTE 3: valor maior

Entrada utilizada: $n = 30$. O programa calcula Fibonacci com recursão simples, recursão em cauda e memoização, registrando os tempos das duas abordagens comparadas diretamente.

EXPLICAÇÃO DAS FUNÇÕES RECURSIVAS

1. RECURSÃO SIMPLES

A função `fibonacciSimples` chama a si mesma para calcular os dois termos anteriores da sequência. Essa implementação é simples de compreender, porém repete vários cálculos, fazendo com que o custo de execução cresça rapidamente à medida que n aumenta.

2. RECURSÃO EM CAUDA

A função `fibonacciCauda` recebe os dois valores anteriores como parâmetros e realiza a próxima chamada com os valores atualizados. Dessa forma, o cálculo avança mantendo o estado necessário na chamada recursiva.

3. RECURSÃO COM MEMOIZAÇÃO

A função `fibonacciMemo` armazena os resultados já calculados em um vetor. Quando um valor já existe no vetor, ele é reutilizado em vez de ser recalculado. Isso reduz significativamente a quantidade de chamadas repetidas da recursão simples.

COMPARAÇÃO DE EFICIÊNCIA

A recursão simples tende a apresentar o pior desempenho para valores maiores de n , pois recalcula repetidamente os mesmos termos da sequência. A memoização elimina grande parte dessas repetições ao armazenar resultados intermediários, reduzindo o número de operações. A recursão em cauda também organiza o cálculo de forma mais eficiente, mantendo apenas os valores necessários para avançar na sequência.

RESPOSTA À QUESTÃO DE AVALIAÇÃO

Por que compreender as diferenças de desempenho entre a recursão simples e uma abordagem otimizada é importante ao escolher um algoritmo recursivo?

Compreender essas diferenças é importante porque duas implementações que produzem o mesmo resultado podem exigir quantidades muito diferentes de tempo e memória. Na recursão simples de Fibonacci, vários valores são recalculados diversas vezes, aumentando rapidamente o número de chamadas e o tempo de processamento. Já uma abordagem otimizada, como a memoização, armazena resultados já obtidos e evita cálculos repetidos. Isso reduz o trabalho computacional e permite lidar com valores maiores de n de forma mais eficiente. Portanto, a escolha da abordagem influencia diretamente o desempenho, o consumo de memória e a capacidade do programa de trabalhar com entradas maiores sem desperdício de recursos.

CHECKLIST DE VALIDAÇÃO

Item de Validação	Status
Função recursiva simples implementada	OK
Versão otimizada com memoização criada	OK
Função de Fibonacci em cauda implementada	OK
Comparação de tempo de execução prevista no programa	OK
Diferentes valores de n utilizados nos testes	OK
Análise de desempenho e consumo de recursos apresentada	OK
Código completo do exercício inserido no relatório	OK
Relatório entregue sem prints de execução, conforme solicitado	OK

CONCLUSÃO

A atividade prática foi desenvolvida com foco na recursividade em C, utilizando três formas de cálculo de Fibonacci: recursão simples, recursão em cauda e recursão otimizada com memoização. A comparação mostra que a organização do algoritmo e o reaproveitamento de resultados podem reduzir significativamente o trabalho necessário para obter a mesma resposta. Dessa forma, a análise de eficiência é fundamental para escolher uma solução adequada ao tamanho das entradas e aos recursos disponíveis.

REFERÊNCIAS BIBLIOGRÁFICAS

ONLINEGDB. Online C Compiler & Debugging Tool. Disponível em:
<https://www.onlinegdb.com/>